Mohammed V University
Faculty of Sciences

Master IGOV

Lab 1
Hash functions and Signature Schemes

Blockchain: Foundations & Applications

Written by:
Hajar Chahbi

Supervised by:
Pr. Yahya Benkaouz

2025–2026

Contents

List of Figures

1 Part 1 – RSA Cryptography in Java

1.1 Review and explaination of the provided RSA implementation

1.1.1 Objective

The objective of this part is to review the provided RSA implementation in Java (JCA/JCE) and explain the purpose of each method: key generation, key loading, encryption/decryption, and signature/verification.

1.1.2 Method explanations

- **`generateKeys()`**: Generates a 2048-bit RSA public/private key pair using `KeyPairGenerator`. The keys are encoded and saved into two files: `public.key` and `private.key`.
- **`loadPublicKey(String filename)`**: Loads the public key from a file by reading its bytes and reconstructing a `PublicKey` object using `X509EncodedKeySpec`.
- **`loadPrivateKey(String filename)`**: Loads the private key from a file by reading its bytes and reconstructing a `PrivateKey` object using `PKCS8EncodedKeySpec`.
- **`encrypt(byte[] message, PublicKey pub)`**: Encrypts a message using the RSA public key with OAEP padding (`RSA/ECB/OAEPWithSHA-256AndMGF1Padding`).
- **`decrypt(byte[] cipherText, PrivateKey priv)`**: Decrypts the ciphertext using the RSA private key and recovers the original plaintext.
- **`sign(byte[] message, PrivateKey priv)`**: Generates a digital signature using `SHA256withRSA`.
- **`verify(byte[] message, byte[] signature, PublicKey pub)`**: Verifies the digital signature and returns `true` if it is valid.
- **`main(String[] args)`**: Executes a complete test of the RSA workflow.

1.1.3 Execution

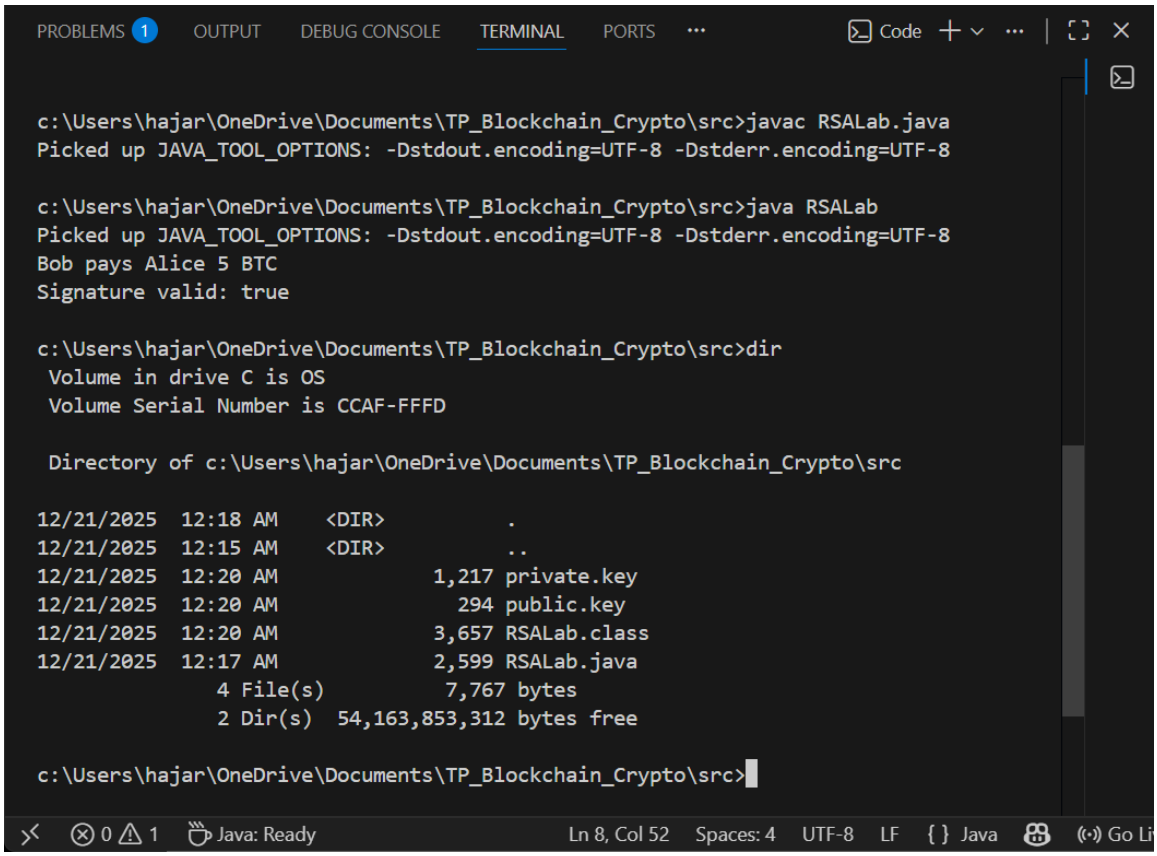

Figure 1: RSA execution output showing the decrypted message, successful signature verification, and the generated key files (`public.key` and `private.key`).

1.2 Client–Server Application Using RSA

1.2.1 Secure File Transfer

The objective of this part is to ensure secure file transmission between a client and a server using RSA encryption.
The client reads the file `message.txt`, encrypts its content using the server's public key, and sends the encrypted data over the network.
After reception, the server decrypts the data using its private key and reconstructs the original file as `received.txt`.
The successful creation of `received.txt` with the expected content confirms the validity of the encryption and decryption process.

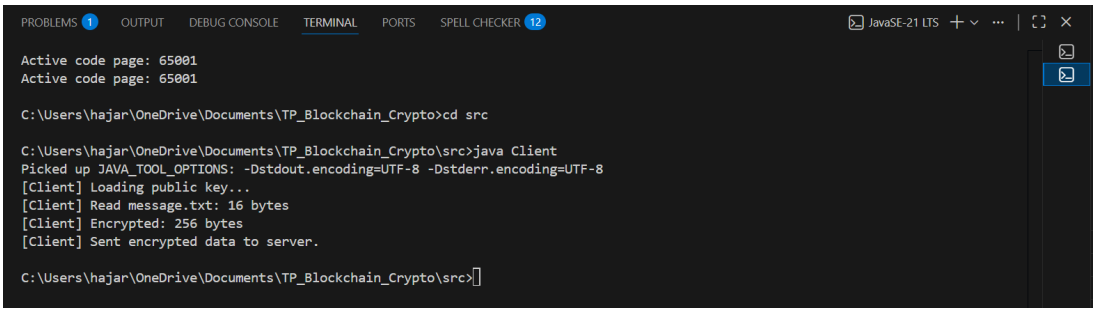

Figure 2: Client-side execution showing file encryption and transmission.

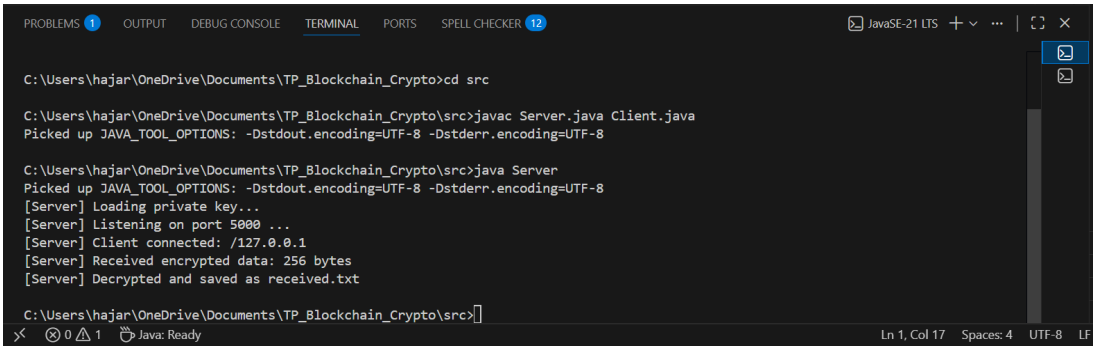

Figure 3: Server-side execution showing file reception and successful decryption.

1.2.2 File Signing and Verification

In this scenario, the client sends a file to the server for authentication. The server generates a digital signature using its RSA private key and sends the signature back to the client. The client verifies the signature using the server's public key, ensuring both authenticity and integrity of the file.

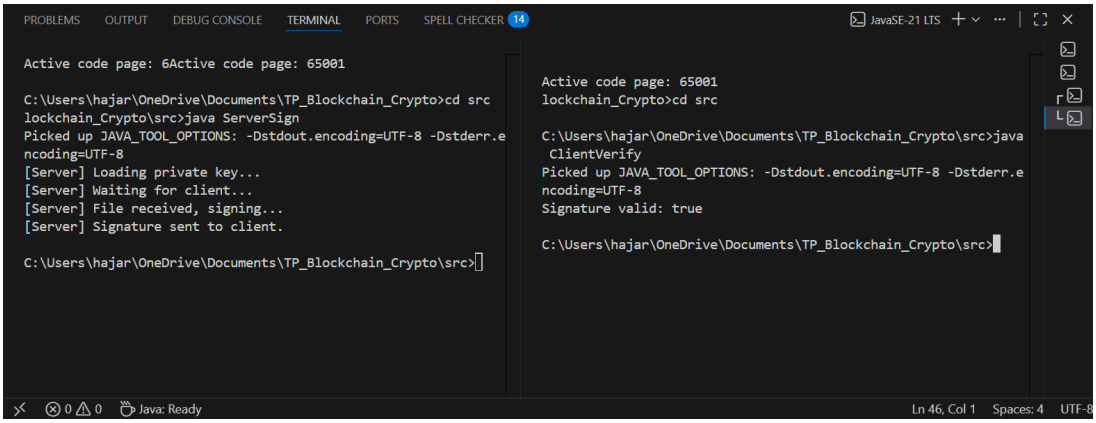

Figure 4: Verification of the digital signature using the RSA public key.

2 Part 2 – Lamport Signature Scheme

2.1 Principle

The Lamport signature scheme is a one-time signature algorithm based on cryptographic hash functions. Unlike RSA, it does not rely on number-theoretic assumptions but on the preimage resistance of hash functions. Each signature key pair can be used only once.

2.2 Key Generation

The Lamport private key consists of two sets of random values. For each possible bit value (0 or 1), 256 random 256-bit values are generated. The public key is obtained by applying a cryptographic hash function (SHA-256) to each element of the private key.

2.3 Signature Generation

To sign a message, the message is first hashed using SHA-256. For each bit of the hash, the signer selects one value from the private key: if the bit is 0, the corresponding value from the first set is chosen; if the bit is 1, the value from the second set is chosen. The collection of selected values forms the Lamport signature.

2.4 Signature Verification

To verify a Lamport signature, the verifier hashes the message again and computes the hash of each element of the received signature. These hashes are then compared with the corresponding values of the public key. If all values match, the signature is considered valid. If the message is modified after signing, the verification fails, demonstrating the integrity property of the scheme.

2.5 Experimental Results

The Lamport signature scheme was tested using a sample message. The verification succeeds for the original message and fails when the message is tampered with, confirming the correctness of the implementation.

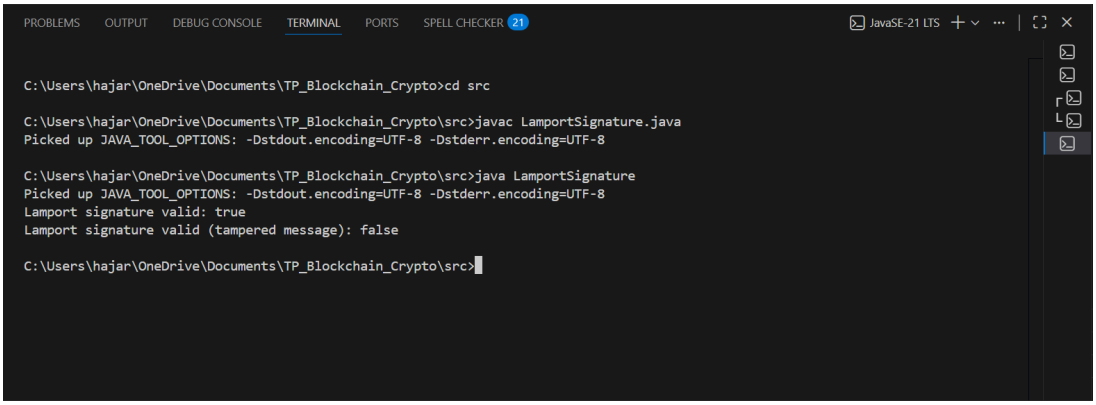

Figure 5: Lamport signature verification showing a valid signature for the original message and failure after message tampering.

2.6 Merkle Tree Signature Scheme Based on Lamport

2.6.1 Principle

A Merkle Tree Signature scheme extends Lamport one-time signatures into a multi-use signature scheme by using a Merkle tree. Instead of publishing a single Lamport public key, it generates N Lamport key pairs and authenticate them using a Merkle tree. The Merkle root becomes the *master public key*.

2.6.2 Key Generation and Merkle Tree Construction

- **Generate N Lamport key pairs:** each key pair consists of a Lamport private key and its corresponding Lamport public key.
- **Compute leaf values:** for each Lamport public key, we compute a leaf hash (e.g., by hashing a serialized representation of the Lamport public key using SHA-256).
- **Build the Merkle tree:** internal nodes are computed by hashing the concatenation of the left and right child hashes. If the number of nodes is odd at a level, the last node is duplicated (standard Merkle tree padding).
- **Publish the Merkle root:** the root hash is published as the **master public key**.

2.6.3 Signature Generation

To sign a message using the key pair at index i:

- Compute a standard Lamport signature on the message using the Lamport private key at index i.
- Include the Lamport public key corresponding to index i so the verifier can recompute the leaf hash.
- Include the **Merkle authentication path** (the list of sibling hashes from the leaf up to the root) required to reconstruct the Merkle root.

Therefore, the complete Merkle–Lamport signature contains:

$$\text{Signature} = (i,\ \sigma_{\text{Lamport}},\ PK_{\text{Lamport}},\ \text{AuthPath}_i)$$

2.6.4 Verification

To verify a Merkle–Lamport signature:

- Verify the Lamport signature σ_{Lamport} using the provided Lamport public key PK_{Lamport}.
- Recompute the leaf hash from PK_{Lamport}.
- Using the authentication path AuthPath_i, reconstruct the Merkle root.
- Accept the signature if (1) the Lamport signature is valid and (2) the reconstructed root matches the published master public key.

2.6.5 Experimental Results

The implementation was tested by generating N Lamport key pairs, building the Merkle tree, signing a sample message using a chosen index, and verifying both: (1) the Lamport signature validity and (2) the correctness of the Merkle authentication path (root match). The outputs confirm a valid Merkle–Lamport signature.

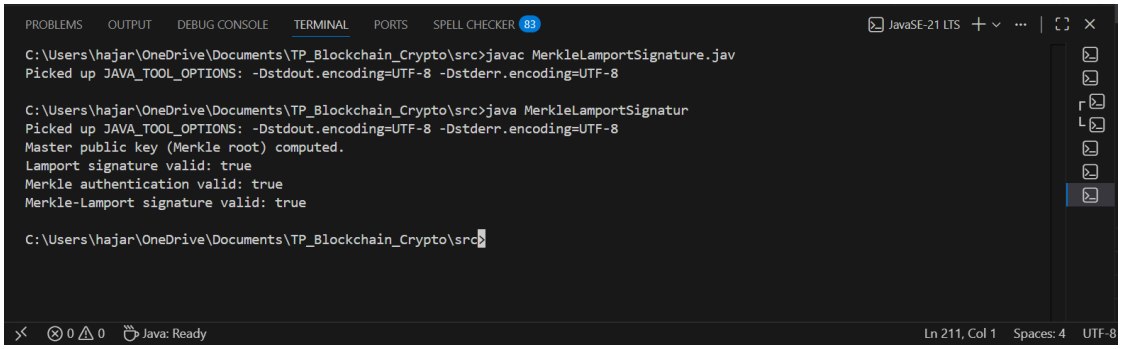

Figure 6: Execution output showing Lamport verification, Merkle authentication validity, and the final Merkle–Lamport signature validity.

3 Part 3 – RSA Blind Signature

3.1 Principle

A blind signature scheme allows an authority to sign a message without learning its content. The requester blinds the message before sending it to the signer, who signs the blinded value. After receiving the signed blinded message, the requester unblinds the signature. The resulting signature can be publicly verified while preserving the privacy of the original message. This mechanism is commonly used in electronic voting and digital cash systems.

3.2 Blind Signature Process

The blind signature protocol based on RSA follows four main steps:

- The message is first hashed using a cryptographic hash function (SHA-256).
- The hash is blinded using a random value and the RSA public key.
- The signer applies the RSA private key to sign the blinded message.
- The requester unblinds the signature and verifies it using the public key.

3.3 Implementation and Verification

In this experiment, an RSA key pair is generated locally.
To support blind-signing of arbitrary data regardless of its type or size, the input data is first hashed using the SHA-256 hash function.
In the code, the direct use of the message was replaced by its hash value ($m = \text{SHA-256}(\text{data})$) **before computing the blinded message** ($m' = m \cdot r^e \bmod n$).

The blinding, signing, and unblinding operations are therefore applied to the hash value rather than the raw data.
After unblinding, the final signature is verified by checking that the RSA verification equation holds.
This modification ensures that the blind signature scheme can sign any type of data while keeping the RSA operations on a fixed-length input.
A successful verification confirms that the blind signature is valid without revealing the original message to the signer.

3.4 Experimental Results

The implementation was tested using a sample message. The results confirm the correctness of the blind signature protocol.

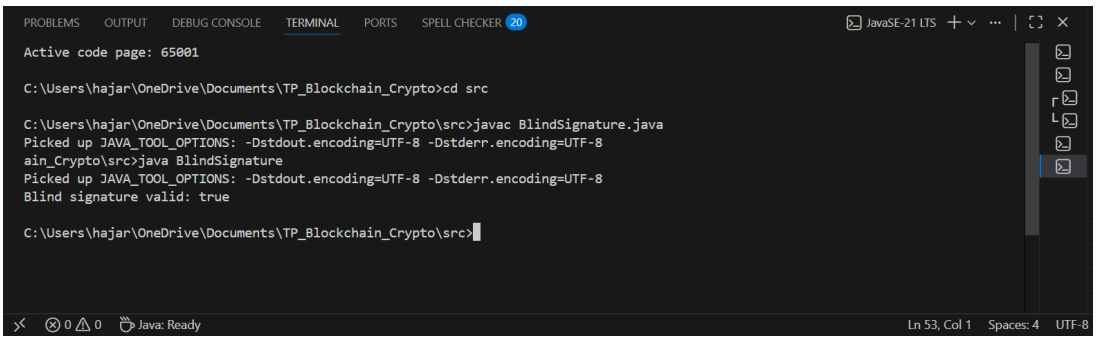

Figure 7: Execution of the RSA blind signature scheme showing successful verification.

4 Conclusion

This laboratory work focused on the practical implementation of cryptographic signature schemes used in blockchain and security-oriented systems.

- **RSA Cryptosystem:** We implemented key generation, encryption/decryption, and digital signatures, and developed a secure client–server application ensuring confidentiality, authenticity, and integrity.
- **Lamport and Merkle-Based Signatures:** We implemented the Lamport one-time signature scheme and demonstrated its correctness through signature verification and message tampering. To overcome its one-time limitation, we extended the scheme using a Merkle tree construction, allowing multiple secure signatures with a single master public key.
- **Blind Signature:** We implemented an RSA-based blind signature scheme enabling an authority to sign messages without learning their content, a property essential for privacy-preserving applications such as electronic voting and digital cash.

Overall, this laboratory provided a practical understanding of modern cryptographic signature schemes and highlighted their importance in building secure and privacy-aware distributed systems.